MBLP131 · Hafta 4 · Ders Notları

İşletim Sistemleri Temelleri

İş Parçacıkları ve İşlemci Zamanlamasının Temelleri

Bir program aynı anda birkaç işi nasıl yapar — ve sırada kimin çalışacağına kim karar verir?

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Bu haftanın sonunda yapabilmeniz gerekenler

1. *İş parçacığını* (thread) tanımlamak; bir sürecin iş parçacıklarının neleri paylaştığını, her birinin neleri kendine sakladığını saymak.
2. Çok iş parçacıklı programlamanın dört yararını ve *eşzamanlılık* (concurrency) ile *paralellik* (parallelism) arasındaki farkı açıklamak.
3. Bir programın N çekirdekteki en büyük hızlanmasını Amdahl yasasıyla hesaplamak.
4. Kullanıcı düzeyi ve çekirdek düzeyi iş parçacıklarını; çoktan bire, birebir ve çoktan çoğa modellerini karşılaştırmak.
5. Küçük Pthreads programlarını C ile okuyup çalıştırmak, iş parçacıklarını `ps` ve `/proc` ile gözlemek.
6. İşlemci–G/Ç patlama döngüsünü, hazır kuyruğunu, dört zamanlama karar noktasını, kesintili zamanlamayı ve dağıtıcıyı anlatmak.
7. Bir Gantt şemasından işlemci kullanımını, üretilen işi, dönüş, bekleme ve yanıt süresini hesaplamak.

İçindekiler

1 Süreçlerden İş Parçacıklarına **1**

1.1 3. haftanın kısa bir tekrarı 1
1.2 Neden iş parçacığı? 2
1.3 İş parçacığı nedir? 2
1.4 Ne paylaşılır, ne özeldir? 3
1.5 Çok iş parçacıklı programlamanın yararları 3

2 Çok Çekirdekli Programlama **4**

2.1 Eşzamanlılık ve paralellik 4

2.2 Programlama zorlukları 4
2.3 Amdahl yasası 5

3 Kullanıcı İş Parçacıkları, Çekirdek İş Parçacıkları ve Çoklu İş Parçacığı Modelleri 6
3.1 Kullanıcı düzeyi ve çekirdek düzeyi iş parçacıkları 6
3.2 Üç model 7

4 İş Parçacığı Kütüphaneleri ve Uygulamalı Pthreads 8
4.1 Örnek 1: iş parçacığı gerçekten süreçten ucuz mu? 8
4.2 Örnek 2: üç iş parçacığından merhaba 9
4.3 Örnek 3: bir diziyi T iş parçacığıyla toplamak 11

5 İşlemci Zamanlaması: Temel Kavramlar 12
5.1 İşlemci–G/Ç patlama döngüsü 12
5.2 Örnek 4: işlemci yoğun ve G/Ç yoğun, ölçülmüş 13
5.3 İşlemci zamanlayıcısı ve hazır kuyruğu 14
5.4 Zamanlayıcı ne zaman karar vermek zorundadır? 14
5.5 Kesintisiz ve kesintili zamanlama 15
5.6 Dağıtıcı 15

6 Zamanlama Ölçütleri 16
6.1 Tanımların formülleri 17
6.2 Çözümlü örnek: geliş sırasına göre dört süreç 18
6.3 İyileştirme hedefleri 18

7 Özet 19

8 Sözlük 19

9 Kendiniz Deneyin 20

10 Tekrar Soruları 21

11 İleriye Bakış: 5. Hafta 22

1 Süreçlerden İş Parçacıklarına

1.1 3. haftanın kısa bir tekrarı

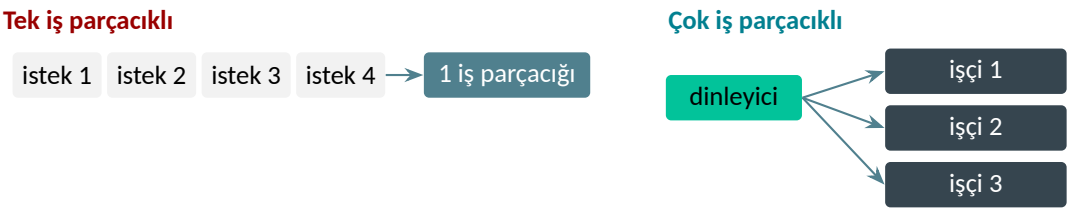
Geçen hafta **süreç** (process) kavramıyla tanıştık: yürütülmekte olan bir program. Sürecin bir bellek görüntüsü (kod/metin bölümü, veri, bss, öbek (heap) ve yığıt (stack)), bir işlemci durumu (program sayacı, yazmaçlar) ve açık dosyalar gibi kaynakları vardır; bunların hepsi **süreç denetim bloğunda** (process control block, PCB) tutulur. Süreç beş durum arasında gezinir (yeni, hazır, çalışıyor, bekliyor, sonlandı); **bağlam değişimi** (context switch) bir sürecin işlemci durumunu PCB'sine kaydedip bir başkasınınkini yükler; `fork()`, `exec()` ve `wait()` süreçleri yaratır, değiştirir ve toplar.

O resimde her süreç aynı anda tam olarak tek bir iş yapar: *tek* bir program sayacı vardır. Bu hafta bu kısıtlamayı kaldırıyoruz.

1.2 Neden iş parçacığı?

Saniyede binlerce istek alan bir web sunucusu düşünün. Bir isteğe hizmet etmek biraz işlemci zamanı ve çok fazla bekleme gerektirir: diski, veritabanını, ağı beklemek.

- **Tek iş parçacıklı** bir sunucu istekleri teker teker işler. 1. istek diski beklerken 2., 3. ve 4. istekler, işlemci boşta olduğu hâlde sırada bekler.
- Sunucu her istek için **yeni bir süreç yaratabilir** (ilk Apache web sunucuları bunu `fork()` ile yapıyordu). Bu çalışır; ama süreç yaratmak pahalıdır ve süreçler belleği paylaşmadığı için ortak bir web sayfası önbelleğini kolayca kullanamazlar.
- **Çok iş parçacıklı** bir sunucuda istekleri kabul eden bir *dinleyici* iş parçacığı vardır; her isteği bir *işçi* iş parçacığına verir. Tüm iş parçacıkları tek bir sürecin içinde yaşar ve onun belleğini paylaşır.



Şekil 1: Tek iş parçacıklı sunucu istekleri tek tek işler; çok iş parçacıklı sunucu her isteğe kendi iş parçacığını verir.

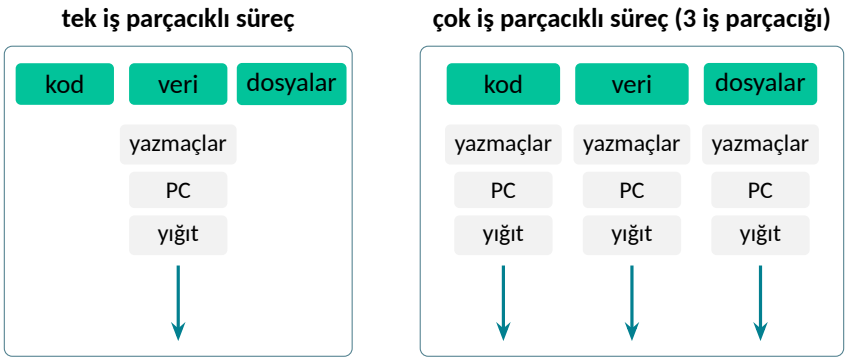
Aynı fikir neredeyse her etkileşimli programda karşımıza çıkar. Bir kelime işlemcide bir iş parçacığı tuş vuruşlarını okur, biri ekranı yeniden çizer, biri yazımı denetler, biri de birkaç dakikada bir yedek kopya kaydeder. Bir tarayıcı sayfaları yükler, resimleri çözer ve betikleri birçok iş parçacığında çalıştırır; bir oyun görüntü, fizik, ses ve ağ kodunu ayrı iş parçacıklarında yürütür. Her durumda iş parçacıkları *aynı* veri (belge, sayfa, oyun dünyası) üzerinde çalışır; dolayısıyla belleği paylaşmaları gerekir.

1.3 İş parçacığı nedir?

Tanım

İş parçacığı (thread), işlemci kullanımının temel birimidir. Bir *iş parçacığı kimliği* (thread ID), bir *program sayacı* (program counter), bir *yazmaç kümesi* (register set) ve bir *yığıttan* (stack) oluşur. Aynı sürecin diğer iş parçacıklarıyla kod bölümünü, veri bölümünü ve açık dosyalar gibi işletim sistemi kaynaklarını paylaşır.

Geleneksel, “ağır siklet” bir süreç, tam olarak tek bir iş parçacığı olan süreçtir. Çok iş parçacıklı bir sürecin birden çok program sayacı vardır: aynı programın birkaç farklı yeri aynı anda yürütülmektedir.



Şekil 2: Nane yeşili: sürecin tüm iş parçacıklarınca paylaşılır. Gri: her iş parçacığına özel bir kopya. Her ok bir yürütme akışıdır (bir iş parçacığı).

3. haftanın diliyle söylersek PCB ikiye bölünür: sürece ait bir kısım (bellek haritası, açık dosyalar, kullanıcı kimliği) ve iş parçacığına ait küçük bir kısım (durum, kaydedilmiş yazmaçlar ve program sayacı, yığıt işaretçisi). Linux her iş parçacığına *görev* (task) der ve her birine kendi çekirdek kaydını verir; bir sürecin iş parçacıkları yalnızca aynı bellek haritasını ve aynı açık dosya tablosunu gösterir. Eski kaynaklar iş parçacıklarına **hafif süreç** (lightweight process, LWP) der; bu kısaltma `ps` çıktısında hâlâ görünür.

1.4 Ne paylaşılır, ne özeldir?

Öğe	İş parçacıkları…	Neden
Kod (metin bölümü)	paylaşır	Tüm iş parçacıkları aynı programın fonksiyonlarını çalıştırır.
Global veri ve öbek	paylaşır	Global bir değişken ya da `malloc` ile ayrılmış bir dizi her iş parçacığınca görülür.
Açık dosyalar, PID, kullanıcı kimliği	paylaşır	Kaynaklar iş parçacığına değil, sürece aittir.
Program sayacı ve yazmaçlar	özel	Her iş parçacığı kodda kendi yerindedir.
Yığıt (yerel değişkenler)	özel	Her iş parçacığının kendi fonksiyon çağrı zinciri vardır.
İş parçacığı kimliği, zamanlama durumu	özel	İşletim sistemi her iş parçacığını ayrı zamanlar.

Paylaşım iş parçacıklarının varlık nedenidir — ve aynı zamanda başlıca tehlikesidir. İki iş parçacığı aynı değişkeni aynı anda güncellerse güncellemeler kaybolabilir. Buna **yarış durumu** (race condition) denir ve 5. haftanın ikinci yarısının ana konusudur.

1.5 Çok iş parçacıklı programlamanın yararları

1. **Yanıt verebilirlik (responsiveness).** Bir iş parçacığı bloke olsa (G/Ç beklese) ya da uzun bir hesap yapsa bile diğer iş parçacıkları çalışmayı sürdürür. Tanıdık “uygulama yanıt vermiyor” mesajı çoğunlukla kullanıcı

arayüzünü yöneten tek iş parçacığının başka bir işle meşgul olduğu anlamına gelir.

2. **Kaynak paylaşımı (resource sharing).** İş parçacıkları süreçlerinin belleğini ve kaynaklarını kendiliğinden paylaşır. Ayrı süreçler iş birliği yapmak için paylaşılan bellek ya da borular (pipe) gibi açık mekanizmalar kullanmak zorundadır.
3. **Ekonomi (economy).** İş parçacığı yaratmak ve aynı sürecin iş parçacıkları arasında geçiş yapmak, süreçlere göre çok daha ucuzdur; çünkü yeni bir adres alanı yaratmak ya da adres alanını değiştirmek gerekmez. Bunu Bölüm 4.1'te ölçüyoruz.
4. **Ölçeklenebilirlik (scalability).** Çok çekirdekli bir işlemcide bir sürecin iş parçacıkları farklı çekirdeklerde paralel çalışabilir. Tek iş parçacıklı bir süreç, makinede kaç çekirdek olursa olsun yalnızca birini kullanabilir.

> **İş parçacıkları neden vazgeçilmez oldu?**
>
> Yaklaşık 2005'e kadar her yeni işlemci kuşağı daha yüksek bir saat frekansında çalışıyordu; böylece eski tek iş parçacıklı programlar kendiliğinden hızlanıyordu. Sonra güç tüketimi ve ısı, saat hızlarının artışını kabaca 3–5 GHz düzeyinde durdurdu. Çip üreticileri fazladan transistörleri bunun yerine tek bir çipe birkaç çekirdek koymak için kullandı. O zamandan beri bir program yeni donanımda ancak birkaç çekirdeği kullanabiliyorsa, yani çok iş parçacıklıysa (ya da birkaç sürece bölünmüşse) hızlanır.

2 Çok Çekirdekli Programlama

2.1 Eşzamanlılık ve paralellik

Bu iki sözcük sıkça karıştırılır.

- Bir sistem, birden çok görevin *ilerlemesini* destekliyorsa **eşzamanlıdır** (concurrent). Tek çekirdekte işletim sistemi bunu birkaç milisaniyede bir iş parçacıkları arasında geçiş yaparak sağlar; insana iş parçacıkları aynı anda çalışıyormuş gibi görünür.
- Bir sistem birden çok görevi *aynı anda* yürütebiliyorsa **paraleldir** (parallel). Bunun için en az iki çekirdek gerekir.



Şekil 3: Tek çekirdekte dört iş parçacığı (paralellik olmadan eşzamanlılık) ve iki çekirdekte (paralellikle birlikte eşzamanlılık).

Paralellik eşzamanlılığı gerektirir, ama tersi doğru değildir. Bu haftanın ölçümlerinde kullanılan test makinesinin iki çekirdeği vardır (`nproc` komutu 2 yazar).

2.2 Programlama zorlukları

Birkaç çekirdeği iyi kullanan programlar yazmak zordur. Genellikle beş zorluk sayılır:

1. **Görevleri belirlemek:** işin bağımsız çalışabilecek parçalarını bulmak.
2. **Denge:** her çekirdeğe benzer miktarda iş vermek; aksi hâlde biri bitirene kadar bazı çekirdekler boşta bekler.
3. **Veriyi bölmek:** veriyi, her iş parçacığı kendi parçası üzerinde çalışacak biçimde ayırmak.
4. **Veri bağımlılığı:** B görevi A görevinin sonucuna ihtiyaç duyuyorsa B, A'yı beklemelidir. İş parçacıkları *eşzamanlanmalıdır* (synchronisation; 5. hafta).
5. **Test ve hata ayıklama:** iş parçacıkları çok sayıda farklı sırayla iç içe geçebilir; bir hata milyonda bir çalıştırmada ortaya çıkabilir.

İki temel paralellik türü vardır. **Veri paralelliğinde** (data parallelism) aynı işlem verinin farklı parçalarına uygulanır; dizi toplama programımız (Bölüm 4.3) bir örnektir. **Görev paralelliğinde** (task parallelism) farklı iş parçacıkları farklı işlemler yapar; yukarıdaki kelime işlemci iş parçacıkları gibi. Gerçek programlar çoğu zaman ikisini karıştırır.

2.3 Amdahl yasası

Neredeyse her programın çekirdekler arasında bölünemeyen bir kısmı vardır: girdiyi okumak, kısmi sonuçları birleştirmek, yanıtı yazdırmak. Gene Amdahl (IBM) 1967'de bu *seri* kısmın, kaç çekirdek kullanılırsa kullanılsın, elde edilebilecek hızlanmayı sınırladığını gözlemledi.

Tek çekirdekteki çalışma süresi 1 olsun (normalize ediyoruz) ve S bu sürenin seri olan kesri olsun, $0 \le S \le 1$. Geri kalan $1 - S$ kesri N çekirdek arasında kusursuzca bölünebilsin. O zaman

$$T(1) = S + (1 - S) = 1, \qquad T(N) = S + \frac{1 - S}{N}.$$

Hızlanma (speedup) eski sürenin yeni süreye oranıdır:

Amdahl yasası

$$\text{hızlanma}(N) = \frac{T(1)}{T(N)} \le \frac{1}{S + \dfrac{1 - S}{N}}, \qquad \lim_{N \to \infty} \text{hızlanma}(N) = \frac{1}{S}.$$

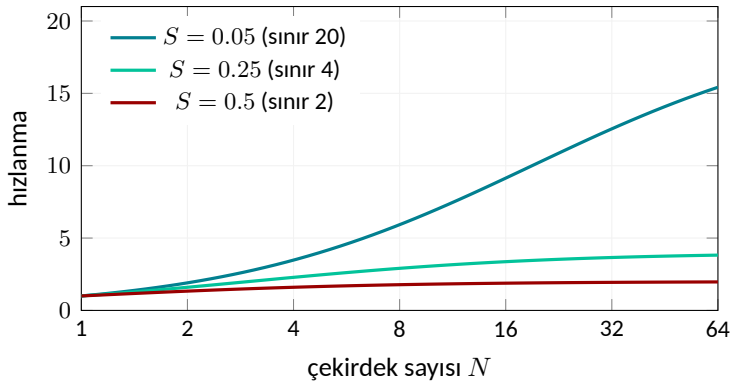
Eşitsizlik işareti bunun en iyi durum olduğunu hatırlatır: gerçek programlar iş parçacığı yaratmanın, onları eşzamanlamanın ve bellek bant genişliğini paylaşmanın bedelini de öder.

%75 paralel, %25 seri

$S = 0.25$ alalım; o hâlde $1 - S = 0.75$.

$$
\begin{aligned}
N = 2 &: \quad \frac{1}{0.25 + 0.75/2} = \frac{1}{0.625} = 1.60, \\
N = 4 &: \quad \frac{1}{0.25 + 0.75/4} = \frac{1}{0.4375} \approx 2.29, \\
N = 8 &: \quad \frac{1}{0.25 + 0.75/8} = \frac{1}{0.34375} \approx 2.91, \\
N = 16 &: \quad \frac{1}{0.25 + 0.75/16} = \frac{1}{0.296875} \approx 3.37, \\
N \to \infty &: \quad \frac{1}{0.25} = 4.
\end{aligned}
$$

Donanımı 8 çekirdekten 16 çekirdeğe çıkararak ikiye katlamak, hızlanmayı yalnızca 2,91'den 3,37'ye yükseltir. Hiçbir çekirdek sayısı bu programı 4 kattan fazla hızlandıramaz.



Şekil 4: Amdahl yasası: hızlanma $1/S$ değerinde düzleşir.

Pratik ders: *önce seri kısmı küçültün, sonra çekirdek ekleyin.* Binlerce çekirdeği olan grafik işlemcileri (GPU) için yazılan programların seri kısmı küçücük tutacak biçimde tasarlanmasının nedeni budur.

3 Kullanıcı İş Parçacıkları, Çekirdek İş Parçacıkları ve Çoklu İş Parçacığı Modelleri

3.1 Kullanıcı düzeyi ve çekirdek düzeyi iş parçacıkları

İş parçacıkları iki düzeyde sağlanabilir.

Kullanıcı düzeyi iş parçacıkları (user-level threads) süreç içindeki bir kütüphane tarafından yönetilir. Çekirdeğin onlardan haberi yoktur: tek iş parçacıklı sıradan bir süreç görür.

+ İş parçacığı yaratmak ve iş parçacıkları arasında geçiş yapmak sıradan fonksiyon çağrılarıdır: çok hızlıdır, sistem çağrısı gerekmez.
− Bir iş parçacığı bloke eden bir sistem çağrısı yaparsa (örneğin yavaş bir aygıttan `read()`), çekirdek bütün süreci bloke eder ve tüm iş parçacıkları durur.

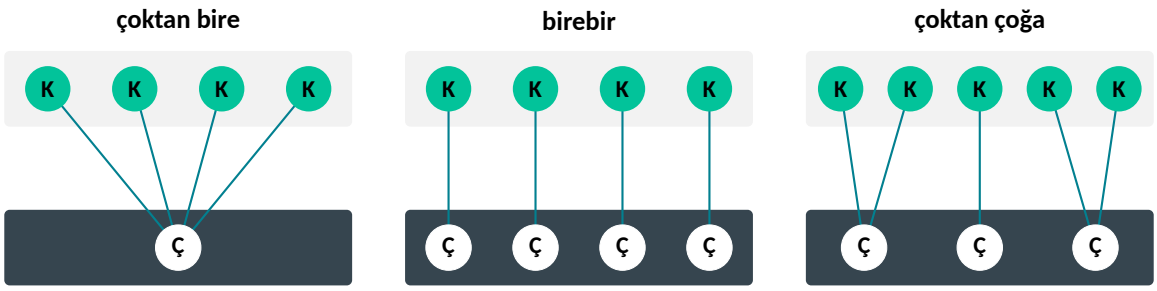
– Çekirdek tek bir varlığı zamanladığı için iş parçacıkları aynı anda iki çekirdekte çalışamaz.

Çekirdek düzeyi iş parçacıkları (kernel-level threads) işletim sistemi çekirdeği tarafından yaratılır, zamanlanır ve yönetilir; her birinin kendi çekirdek kaydı vardır.

+ Bir iş parçacığı bloke olursa çekirdek aynı sürecin başka bir iş parçacığını çalıştırır.
+ Bir sürecin iş parçacıkları birkaç çekirdekte paralel çalışabilir.
– İş parçacığı yaratmak ve geçiş yapmak sistem çağrısı gerektirir ve kullanıcı iş parçacıklarına göre daha yavaştır.

Her kullanıcı iş parçacığı eninde sonunda bir çekirdek iş parçacığı üzerinde çalışmalıdır. Kullanıcı iş parçacıklarının çekirdek iş parçacıklarına eşlenme biçimine **çoklu iş parçacığı modeli** (multithreading model) denir.

3.2 Üç model



Şekil 5: Üç çoklu iş parçacığı modeli. Açık bant: kullanıcı iş parçacıklarının (K) bulunduğu kullanıcı alanı; koyu bant: çekirdek iş parçacıklarının (Ç) bulunduğu çekirdek alanı. (İngilizce kaynaklarda U = user, K = kernel kullanılır.)

Çoktan bire (many-to-one). Birçok kullanıcı iş parçacığı tek bir çekirdek iş parçacığına eşlenir; iş parçacığı yönetimini kullanıcı alanındaki bir kütüphane yapar. Hızlıdır; ama tek bir bloke eden sistem çağrısı tüm iş parçacıklarını bloke eder ve aynı anda yalnızca bir iş parçacığı çekirdekte olabildiğinden model birden çok çekirdeği kullanamaz. Örnekler: Solaris Green Threads, GNU Portable Threads. Bugün çok az sistem kullanır.

Birebir (one-to-one). Her kullanıcı iş parçacığı kendi çekirdek iş parçacığına eşlenir. Bloke eden bir çağrı yalnızca çağıran iş parçacığını bloke eder ve iş parçacıkları çok çekirdekli işlemcilerde paralel çalışır. Bedeli, her kullanıcı iş parçacığının bir çekirdek iş parçacığı gerektirmesidir: iş parçacığı yaratmak bir sistem çağrısı ister ve çok büyük sayıda iş parçacığı çekirdeğe yük olur. **Linux ve Windows birebir modeli kullanır.** Linux'ta `pthread_create()` `clone()` sistem çağrısını çağırır ve her iş parçacığı çekirdekte kendi iş parçacığı kimliğini (TID) alır.

Çoktan çoğa (many-to-many). Birçok kullanıcı iş parçacığı, daha az ya da eşit sayıda çekirdek iş parçacığı üzerine çoğullanır. Program istediği kadar kullanıcı iş parçacığı yaratabilir ve bunlar yine paralel çalışabilir; bloke eden bir çağrı her şeyi durdurmaz. Bedeli karmaşıklıktır: kütüphanedeki zamanlayıcı ile çekirdekteki zamanlayıcının iş birliği yapması gerekir. Yaygın bir türev olan *iki düzeyli model* (two-level model), bir kullanıcı iş parçacığının kendi çekirdek iş parçacığına bağlanmasına da izin verir. İşletim sistemi çekirdekleri bu modeli bırakıp birebir modele geçti; ama dil çalışma ortamları onu yeniden canlandırdı: Go'nun *goroutine*'leri ve

Java'nın *sanal iş parçacıkları* (virtual threads; Java 21, 2023), milyonlarca hafif görevi birkaç çekirdek iş parçacığı üzerinde çalıştırır.

Model	Kullanıcı : çekirdek	Paralel mi?	Bloke eden çağrı durdurur…	Kullananlar
Çoktan bire	$N:1$	hayır	tüm iş parçacıklarını	Green threads (eski)
Birebir	$1:1$	evet	yalnızca o iş parçacığını	Linux, Windows
Çoktan çoğa	$N:M, M \leq N$	evet	yalnızca o iş parçacığını	Go, Java sanal iş parçacıkları

4 İş Parçacığı Kütüphaneleri ve Uygulamalı Pthreads

Bir **iş parçacığı kütüphanesi** (thread library), programcıya iş parçacığı yaratmak ve yönetmek için bir API sunar. Üç ana kütüphane vardır: **POSIX Pthreads** (Linux, macOS ve diğer UNIX sistemleri), **Windows iş parçacığı API'si** ve **Java iş parçacıkları**. Linux ve Windows'ta Java iş parçacıkları, Java sanal makinesi tarafından çekirdek iş parçacıklarına birebir eşlenir.

Pthreads bir gerçekleme değil, bir *belirtimdir* (specification; POSIX standardının parçası). Linux'ta C kütüphanesinde (glibc, NPTL adıyla) `clone()` sistem çağrısının üzerine gerçeklenmiştir. Temel çağrılar şunlardır:

Çağrı	Ne yapar?
`pthread_create(&t, NULL, f, arg)`	`f(arg)` fonksiyonunu çalıştıran yeni bir iş parçacığı başlatır; tutamacı `t`'ye yazılır.
`pthread_join(t, &result)`	`t` iş parçacığı bitene kadar bekler (süreçlerdeki `wait()` gibi).
`pthread_exit(value)`	Çağıran iş parçacığını sonlandırır (`f`'den dönmek de aynı işi görür).
`pthread_self()`	Çağıran iş parçacığının kendi tutamacını döndürür.
`gcc -pthread prog.c`	Pthreads kütüphanesiyle derler ve bağlar.

Bir iş parçacığı fonksiyonu her zaman `void *f(void *arg)` biçimindedir: genel bir işaretçi alır ve genel bir işaretçi döndürür. Birden çok değer geçirmek için bir `struct`'a işaretçi verin.

Aşağıdaki programlar bu notların yanındaki `code/` klasöründe, bir `Makefile` ile birlikte bulunur (`make` yazmanız yeterli). Programlar `gcc -Wall -O2 -pthread` ile derlendi ve başka işlerle paylaşılan iki çekirdekli bir Linux sanal makinesinde çalıştırıldı. 2. haftada olduğu gibi bu derste C yazmanız beklenmiyor: açıklamaları okuyun ve programın ne yaptığına odaklanın.

4.1 Örnek 1: iş parçacığı gerçekten süreçten ucuz mu?

```c
#define N 5000
void *nothing(void *arg) { return arg; }

```

```c
/* 1) N süreç */
for (int i = 0; i < N; i++) {
    pid_t pid = fork();           /* süreci kopyala   */
    if (pid == 0) _exit(0);       /* çocuk çıkar      */
    waitpid(pid, NULL, 0);
}
/* 2) N iş parçacığı */
for (int i = 0; i < N; i++) {
    pthread_t t;
    pthread_create(&t, NULL, nothing, NULL);
    pthread_join(t, NULL);
}
/* programın tamamı: code/create_cost.c */
```

```
$ ./create_cost
fork + wait          :  205.5 us per process
pthread_create + join:   51.9 us per thread
a thread is about 4 times cheaper to create
```

(us mikrosaniye, yani μs demektir.) `fork()` yeni bir adres alanı yaratmak zorundadır: bellek sayfalarının kendisi biri yazana kadar yazınca kopyala (copy-on-write) yöntemiyle paylaşılsa bile sayfa tablolarını ve birçok çekirdek yapısını kopyalar. Yeni bir iş parçacığı ise yalnızca bir yığıt ve küçük bir çekirdek kaydı ister. Hızlı bir fiziksel makinede iş parçacığı yaratmak çoğu zaman yalnızca 10–20 μs sürer ve oran daha büyüktür; ders kitapları 10 ile 30 arasında çarpanlar aktarır. Bizim yaklaşık 4 olan oranımız, bazı işlemlerin daha yavaş olduğu bir sanal makineden gelmektedir.

4.2 Örnek 2: üç iş parçacığından merhaba

```c
/* thread_hello.c -- tek süreç, birkaç iş parçacığı.
 * Her iş parçacığı süreç kimliğini (PID, tüm iş parçacıklarında aynı)
 * ve kendi iş parçacığı kimliğini (TID, her birine özel) yazdırır.
 * Derleme:   gcc -Wall -O2 -pthread thread_hello.c -o thread_hello
 * Çalıştır:  ./thread_hello
 */
#define _GNU_SOURCE
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>        /* getpid(), gettid(), sleep() */

#define NTHREADS 3

int shared_counter = 0;    /* global: TEK kopya, tüm iş parçacıkları görür */

/* Her iş parçacığı hayatına bu fonksiyonda başlar. */
void *worker(void *arg)
{
    int my_number = *(int *)arg;   /* yerel: bu iş parçacığının KENDİ yığıtında */
    printf("thread %d: PID = %d, TID = %d\n",
           my_number, getpid(), gettid());
    shared_counter++;              /* (güvenli değil! bkz. 5. hafta) */
```

```
    sleep(2);                       /* ps ile izleyebilmek için canlı kal */
    return NULL;
}

int main(void)
{
    pthread_t tid[NTHREADS];        /* iş parçacığı "tutamaçları"  */
    int number[NTHREADS];

    printf("main    : PID = %d, TID = %d\n", getpid(), gettid());
    for (int i = 0; i < NTHREADS; i++) {
        number[i] = i + 1;
        /* worker(&number[i]) fonksiyonunu çalıştıran bir iş parçacığı yarat */
        pthread_create(&tid[i], NULL, worker, &number[i]);
    }
    for (int i = 0; i < NTHREADS; i++)
        pthread_join(tid[i], NULL);  /* i. iş parçacığının bitmesini bekle */

    printf("main    : all threads done, shared_counter = %d\n",
           shared_counter);
    return 0;
}
```

```
$ gcc -Wall -O2 -pthread thread_hello.c -o thread_hello
$ ./thread_hello
main    : PID = 6351, TID = 6351
thread 1: PID = 6351, TID = 6352
thread 3: PID = 6351, TID = 6354
thread 2: PID = 6351, TID = 6353
main    : all threads done, shared_counter = 3
```

Üç gözlem:

- Tüm iş parçacıkları *aynı* PID'yi yazar: tek bir sürece aittirler. Her birinin *kendi* TID'si vardır, çünkü her biri ayrı bir çekirdek iş parçacığıdır (birebir model). Ana iş parçacığının TID'si PID'ye eşittir.
- 3. iş parçacığı 2. iş parçacığından önce yazdı. Sırayı zamanlayıcı belirler ve sıra çalıştırmadan çalıştırmaya değişir; bir program buna asla güvenmemelidir.
- `shared_counter` 3 değeriyle biter: global değişken paylaşılır. Ama `shared_counter++`, iş parçacıkları aynı anda çalışırken güvenli değildir (aslında "yükle, topla, sakla" adımlarından oluşur). Üç artırmayla hata neredeyse hiç görünmez; milyonlarca artırmada görünür. Bunu 5. haftada düzelteceğiz.

İş parçacıklarını görmek. Her işçi iki saniye uyur; bu da bize çalışan sürece bakmak için zaman kazandırır. `ps` komutunun `-L` seçeneği her iş parçacığı için bir satır yazar (bazı sütunlar çıkarıldı):

```
$ ./thread_hello &
$ ps -eLf | grep thread_hello
UID    PID  PPID   LWP  C NLWP CMD
root 27241 27234 27241  0    4 ./thread_hello
root 27241 27234 27243  0    4 ./thread_hello
root 27241 27234 27244  0    4 ./thread_hello
```

```
root 27241 27234 27245  0    4 ./thread_hello
$ ls /proc/27241/task
27241  27243  27244  27245
$ grep Threads /proc/27241/status
Threads:   4
```

PID her satırda aynıdır; LWP (lightweight process, hafif süreç) iş parçacığı kimliği, NLWP ise iş parçacığı sayısıdır: ana iş parçacığı artı üç işçi. `/proc/<pid>/task/` dizini her iş parçacığı için bir alt dizin içerir. Başka araçlar: `top -H` ya da `htop` iş parçacıklarını etkileşimli olarak gösterir; Windows'ta Görev Yöneticisi → *Ayrıntılar* sekmesine bir *İş parçacıkları* sütunu eklenebilir.

4.3 Örnek 3: bir diziyi T iş parçacığıyla toplamak

```c
int  *a;             /* büyük dizi: ortak      */
long  partial[MAXT];  /* her iş parçacığına bir */

void *sum_slice(void *arg)
{
    long k = (long)arg;              /* dilimim */
    long start = k * N / T, end = (k + 1) * N / T;
    long s = 0;
    for (long i = start; i < end; i++)
        s + = a[i];
    partial[k] = s;                  /* gözüm   */
    return NULL;
}

/* main() içinde: */
for (long k = 0; k < T; k++)
    pthread_create(&th[k], NULL, sum_slice, (void *)k);
for (long k = 0; k < T; k++) {
    pthread_join(th[k], NULL);
    total + = partial[k];
}
/* tam program (zaman ölçümü, 20 tekrar): code/array_sum.c */
```

$N = 10^8$ tamsayıdan (400 MB) oluşan dizi T eşit dilime bölünür. k. iş parçacığı, indisleri kN/T'den $(k+1)N/T$'ye kadar (bu son değer hariç) olan elemanları kendi gözü olan `partial[k]`'ya toplar. Hiçbir iki iş parçacığı aynı değişkene yazmadığı için kilit gerekmez. Ana iş parçacığı her iş parçacığını `pthread_join()` ile bekler ve kısmi toplamları ekler. Tam program `code/array_sum.c` ayrıca süreyi `clock_gettime()` ile ölçer ve ölçülen süre kararlı olacak kadar uzun olsun diye diziyi 20 kez toplar; doğru sonuç $10^7 \times (0+1+\cdots+9) \times 20 = 10^7 \times 45 \times 20 = 9 \times 10^9$'dur ve her çalıştırma bu değeri verdi.

```
$ nproc
2
$ ./array_sum 1
threads = 1   sum = 9000000000   time = 1.686 s
$ ./array_sum 2
threads = 2   sum = 9000000000   time = 0.999 s
$ ./array_sum 4
threads = 4   sum = 9000000000   time = 0.855 s
```

Makine başka işlerle paylaşıldığı için tek tek çalıştırmalar yaklaşık $\pm 30\%$'a kadar değişti. Bu yüzden her durumu 9 kez çalıştırıp ortancayı (medyan) veriyoruz:

İş parçacığı T	Ortanca süre	En iyi süre	Hızlanma (ortanca)
1	1.564 s	1.498 s	1.00
2	0.932 s	0.761 s	$1.564/0.932 \approx 1.68$
4	0.855 s	0.728 s	$1.564/0.855 \approx 1.83$

Yorum. İki iş parçacığı 1,68 kat hızlanma sağlar; iyi, ama ideal değer olan 2'nin altında. Dört iş parçacığı pek daha iyi değildir, çünkü makinenin yalnızca iki çekirdeği vardır: herhangi bir anda en çok iki iş parçacığı çalışabilir. (Meşgul bir makinede dört iş parçacığı, yalnızca süreç işlemci zamanından daha büyük pay aldığı için ikiden biraz daha hızlı olabilir; ama çekirdek sayısının izin verdiğinden asla daha hızlı olamaz.) Ölçümü Amdahl yasası anlatıyormuş gibi davranırsak S'yi çözebiliriz:

$$1.68 = \frac{1}{S + (1-S)/2} \implies S + \frac{1-S}{2} = \frac{1}{1.68} \implies S = \frac{2}{1.68} - 1 \approx 0.19.$$

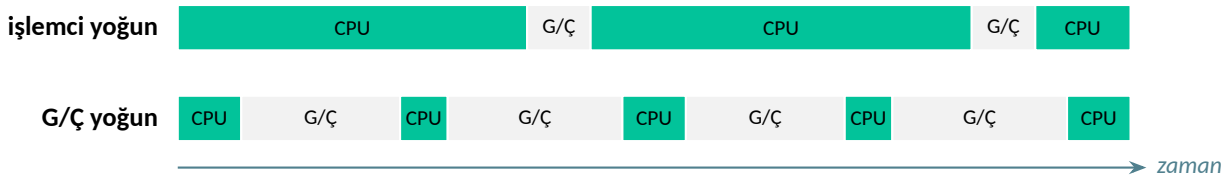
Buradaki "seri kısım" gerçekte bölünemeyen kod değildir: iş parçacıklarını yaratma ve bekleme süresi, iki çekirdeğin aynı bellek bant genişliği için yarışması (tamsayı toplamayı aritmetik değil, bellek hızı sınırlar) ve makinedeki diğer işlerdir. Amdahl yasası kesin bir öngörü değil, bir model ve bir üst sınırdır.

5 İşlemci Zamanlaması: Temel Kavramlar

Tek çekirdekli bir makinede aynı anda yalnızca bir iş parçacığı çalışabilir; N çekirdekte en çok N. Çalışmaya hazır diğer tüm iş parçacıkları beklemek zorundadır. Bir çekirdek her boşaldığında işletim sistemi sırada hangi iş parçacığının çalışacağını seçmelidir. Bu seçime **işlemci zamanlaması** (CPU scheduling) denir. Ders kitapları "süreç zamanlaması"ndan söz eder; Linux ve Windows'ta çekirdek aslında çekirdek iş parçacıklarını zamanlar. Ama aşağıdaki her şey ikisi için de geçerlidir ve biz ders kitabını izleyerek "süreç" diyoruz.

5.1 İşlemci--G/Ç patlama döngüsü

Sürecin yürütülmesi, işlemcide çalışma ile G/Ç bekleme arasında bir *döngüden* oluşur. Süreç bir süre çalışır (bir **CPU patlaması**, CPU burst), sonra G/Ç bekler (bir **G/Ç patlaması**, I/O burst), sonra yeniden çalışır ve bu, son bir CPU patlaması sonlanma isteğiyle bitene kadar sürer.



Şekil 6: İşlemci yoğun bir sürecin az sayıda uzun CPU patlaması, G/Ç yoğun bir sürecin ise çok sayıda kısa CPU patlaması vardır.

- **İşlemci yoğun** (CPU-bound) bir süreç zamanının çoğunu hesaplamayla geçirir: video kodlama, derleme, bilimsel benzetim.
- **G/Ç yoğun** (I/O-bound) bir süreç zamanının çoğunu G/Ç bekleyerek geçirir ve çok sayıda kısa CPU patlaması vardır: metin düzenleyici, web sunucusu, dosya kopyalama.

Gerçek sistemlerde yapılan ölçümler, CPU patlamalarının uzunluklarının tipik bir örüntü izlediğini gösterir: çok sayıda kısa patlama (birkaç milisaniye) ve az sayıda uzun patlama. Dağılım kabaca üsteldir. Zamanlama algoritmaları bu gerçeğe göre tasarlanır (5. hafta). Süreç G/Ç beklerken işlemci boşta kalırdı — zamanlayıcı onu başka bir sürece vermedikçe: 1. haftadaki çoklu programlama fikri.

5.2 Örnek 4: işlemci yoğun ve G/Ç yoğun, ölçülmüş

```
/* ./burst cpu : tek, uzun bir CPU patlaması */
for (long i = 1; i <= 400000000L; i++)
    x + = 1.0 / i;

/* ./burst io : 1000 x (kısa CPU + G/Ç beklemesi) */
for (int i = 0; i < 1000; i++) {
    buf[0] = 'A' + i % 26;     /* CPU    */
    write(fd, buf, 4096);      /* G/Ç... */
    fsync(fd);                 /* ...diski bekle */
}
/* programın tamamı: code/burst.c */
```

Kabuktaki `time` komutu üç sayı raporlar: `real` geçen duvar saati süresidir; `user` ve `sys` sürecin kullanıcı modunda ve çekirdek modunda tükettiği işlemci zamanıdır.

```
$ time ./burst cpu
cpu: harmonic sum = 20.384191
real    0m1.266s
user    0m1.197s
sys     0m0.001s
$ time ./burst io
io: wrote 1000 blocks of 4 KB, each followed by fsync
real    0m0.384s
user    0m0.000s
sys     0m0.104s
```

Süreç zamanın ne kadarında bekledi?

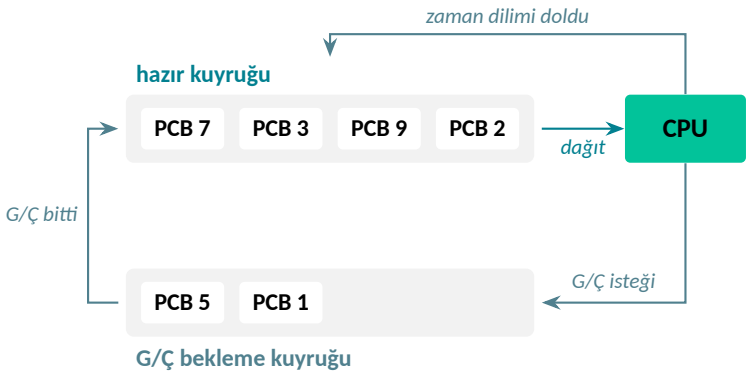
İşlemci yoğun çalıştırmada user + sys $= 1.198$ s $\approx$ real $= 1.266$ s: süreç neredeyse tüm zaman boyunca işlemciyi kullandı. G/Ç yoğun çalıştırmada

$$\frac{\text{user} + \text{sys}}{\text{real}} = \frac{0.000 + 0.104}{0.384} \approx 0.27,$$

yani süreç işlemciyi zamanın yalnızca %27'sinde kullandı; zamanın yaklaşık %73'ünde `fsync()` içinde bloke olmuş, diski bekliyordu. Bu süre boyunca işlemci diğer süreçlere açıktı. Ayrıca işlemci zamanının neredeyse tamamının `sys` olduğuna dikkat edin: `write()` ve `fsync()` sistem çağrılarıdır. (Harmonik toplam da tutarlıdır: $\sum_{i=1}^{n} 1/i \approx \ln n + 0.5772$ ve $\ln(4 \times 10^8) + 0.5772 \approx 20.384$.)

5.3 İşlemci zamanlayıcısı ve hazır kuyruğu

İşlemci her boşa çıktığında işletim sistemi **hazır kuyruğundaki** (ready queue) süreçlerden birini çalıştırmak üzere seçer. İşletim sisteminin bu seçimi yapan kısmına **kısa vadeli zamanlayıcı** (short-term scheduler) ya da **işlemci zamanlayıcısı** (CPU scheduler) denir.

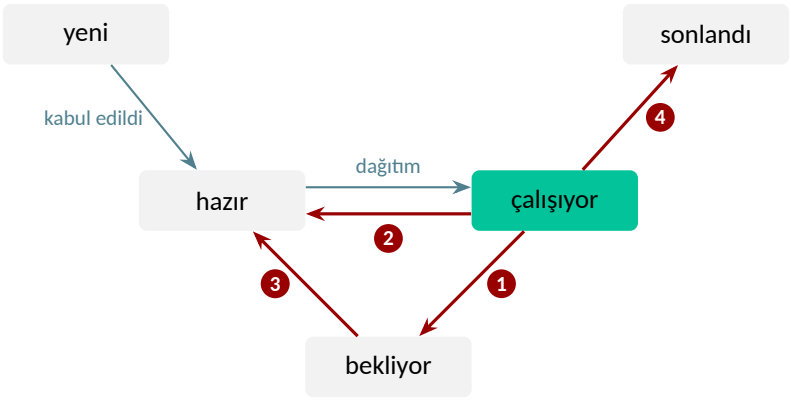


Şekil 7: Süreçler hazır kuyruğu, işlemci ve aygıt bekleme kuyrukları arasında hareket eder.

- Hazır kuyruğu, 3. hafta diyagramındaki *hazır* durumunda olan süreçlerin PCB'lerini tutar: tam olarak şu anda çalışabilecek süreçlerin.
- Hazır kuyruğu mutlaka ilk giren ilk çıkar (FIFO) olmak zorunda değildir. Bir öncelik kuyruğu, bir ağaç ya da sırasız bir liste olabilir; süreçlerin kuyruktan alınma sırası **zamanlama algoritmasıdır** (scheduling algorithm; 5. hafta). Linux hazır görevlerini dengeli bir ikili ağaçta (kırmızı-siyah ağaç, red–black tree) tutar.
- Kısa vadeli zamanlayıcı çok sık çalışır — birkaç milisaniyede bir — bu yüzden hızlı olmalıdır.

5.4 Zamanlayıcı ne zaman karar vermek zorundadır?

Zamanlama kararları, hepsi süreç durum diyagramında görülebilen dört durumda verilebilir:



Şekil 8: Bir zamanlama kararının verilebileceği dört geçiş.

1. Süreç *çalışıyor* durumundan *bekliyor* durumuna geçer: G/Ç ister ya da bir çocuk süreç için `wait()` çağırır.
2. Süreç *çalışıyor* durumundan *hazır* durumuna geçer: bir kesme olur; örneğin zamanlayıcı saati (timer) zaman diliminin bittiğini bildirir.
3. Süreç *bekliyor* durumundan *hazır* durumuna geçer: G/Ç'si tamamlanmıştır.
4. Süreç *sonlanır*.

1. ve 4. durumlarda seçenek yoktur: işlemci boşalmıştır ve (hazır kuyruğu boş değilse) yeni bir süreç *seçilmelidir*. 2. ve 3. durumlarda bir seçenek vardır: işletim sistemi o anki sürecin devam etmesine izin *verebilir* ya da başka bir sürece geçebilir — örneğin az önce hazır hâle gelen daha önemli bir sürece.

5.5 Kesintisiz ve kesintili zamanlama

- **Kesintisiz** (non-preemptive) ya da *iş birlikçi* (cooperative) zamanlamada zamanlayıcı yalnızca 1. ve 4. durumlarda çalışır. Bir süreç işlemciyi aldığında, bloke olana ya da sonlanana kadar elinde tutar. Bu basittir; ama tek bir uzun ya da hatalı süreç diğer herkesi bekletebilir. Windows 3.x ve klasik Mac OS (OS X öncesi) böyle çalışıyordu: sonsuz döngüye takılan bir program bütün bilgisayarı dondururdu.
- **Kesintili** (preemptive) zamanlamada zamanlayıcı 2. ve 3. durumlarda da çalışabilir: işletim sistemi zamanlayıcı kesmesini kullanarak işlemciyi çalışan bir süreçten geri alabilir. Tüm modern genel amaçlı işletim sistemleri — Windows, macOS, Linux, Android, iOS — kesintilidir.

Kesinti (preemption) daha iyi yanıt verebilirlik sağlar ve sistemi kontrolden çıkan programlardan korur; ama bir bedeli vardır: bir süreç *herhangi bir* komutta, hatta başka bir süreç ya da iş parçacığıyla paylaştığı veriyi güncellemenin ortasında kesilebilir. Yarış durumları böyle ortaya çıkar ve 5. hafta bu yüzden eşzamanlama ile devam eder.

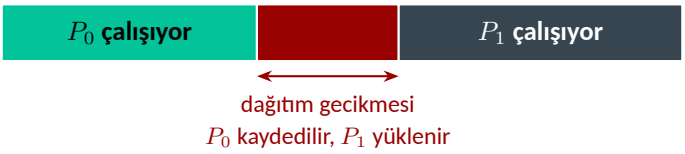
5.6 Dağıtıcı

Zamanlayıcı sırada *hangi* sürecin çalışacağına karar verir; **dağıtıcı** (dispatcher) ise işlemciyi o sürece fiilen veren modüldür. Üç görevi vardır:

1. bağlamı değiştirmek: eski sürecin durumunu PCB'sine kaydetmek ve yenisinin durumunu yüklemek;
2. kullanıcı moduna geçmek;

3. yeni programda doğru yere (kaydedilmiş program sayacına) atlamak.

Dağıtıcının bir süreci durdurup başka birini başlatması için gereken süreye **dağıtım gecikmesi** (dispatch latency) denir. Saf ek yüktür: bu süre içinde yararlı hiçbir iş yapılmaz.



Şekil 9: Dağıtım gecikmesi (ölçekli değildir: gecikme mikrosaniyeler, zaman dilimi milisaniyeler mertebesindedir).

Dağıtım gecikmesi küçüktür, tipik olarak birkaç mikrosaniye; ama çok sık yaşanır. `vmstat 1` komutu, başka bilgilerin yanı sıra saniyedeki bağlam değişimi sayısını `cs` sütununda yazar. Başka iş yapmayan test makinemizde art arda iki bir saniyelik örnek 487 ve 614 gösterdi; meşgul bir sunucu kolayca on binlere ulaşır.

Gönüllü ve zorla bağlam değişimleri. Linux, bir sürecin işlemciyi bloke olduğu için kendisinin kaç kez bıraktığını (*gönüllü*, voluntary geçiş; 1. durum) ve zamanlayıcının işlemciyi kaç kez elinden aldığını (*zorla*, involuntary geçiş; 2. durum) ayrı ayrı sayar. GNU `time` programı `-v` seçeneğiyle ikisini de raporlar (kullanıcı ve sistem süreleri dahil diğer satırlar çıkarıldı):

```
$ /usr/bin/time -v ./burst cpu
  Elapsed (wall clock) time (h:mm:ss or m:ss): 0:01.22
  Voluntary context switches: 1
  Involuntary context switches: 50
$ /usr/bin/time -v ./burst io
  Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.46
  Voluntary context switches: 3714
  Involuntary context switches: 0
```

İşlemci yoğun program neredeyse hiç bloke olmadı, ama 1,22 s içinde 50 kez kesildi: ortalama yaklaşık 24 ms'de bir. G/Ç yoğun program hiç kesilmedi, ama 3714 kez bloke oldu (her `fsync()` birden fazla kez bekleyebilir, örneğin dosya sistemi günlüğü (journal) için). Aynı sayaçlar, çalışan bir süreç için `/proc/<pid>/status` dosyasında da görülebilir; ilgili alanlar şunlardır: `voluntary_ctxt_switches` ve `nonvoluntary_ctxt_switches`.

6 Zamanlama Ölçütleri

Farklı zamanlama algoritmaları farklı süreçleri kayırır. Onları karşılaştırmak için ölçülebilir ölçütlere ihtiyacımız var.

Ölçüt	Tanım	Hedef
İşlemci kullanımı (CPU utilisation)	İşlemcinin yararlı işle meşgul olduğu zamanın oranı; gerçek bir sistemde yaklaşık %40'tan (hafif yük) %90'a (ağır yük) kadar.	en büyük
Üretilen iş (throughput)	Birim zamanda tamamlanan süreç sayısı.	en büyük
Dönüş süresi (turnaround time)	Bir sürecin gelişinden (sunuluşundan) tamamlanmasına kadar geçen süre; hazır kuyruğunda bekleme, çalışma ve G/Ç dahil.	en küçük
Bekleme süresi (waiting time)	Sürecin *hazır kuyruğunda* beklediği toplam süre.	en küçük
Yanıt süresi (response time)	Gelişten, sürecin işlemciyi ilk kez aldığı ana (ilk yanıtına) kadar geçen süre.	en küçük

Zamanlama algoritması bir sürecin ne kadar hesap yaptığını ya da G/Ç'sinin ne kadar sürdüğünü değiştirmez; yalnızca sürecin hazır kuyruğunda ne kadar oturduğunu değiştirir. Zamanlayıcıdan en doğrudan etkilenen ölçütün *bekleme süresi* olmasının nedeni budur.

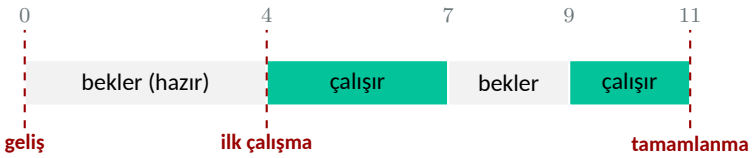
6.1 Tanımların formülleri

Geliş zamanı $T_{\text{geliş}}$, toplam CPU patlama süresi T_{patlama}, işlemciyi ilk aldığı an $T_{\text{ilk çalışma}}$ ve tamamlanma zamanı $T_{\text{tamamlanma}}$ olan ve hiç G/Ç yapmayan bir süreç için:

$$\begin{aligned} T_{\text{dönüş}} &= T_{\text{tamamlanma}} - T_{\text{geliş}}, \\ T_{\text{bekleme}} &= T_{\text{dönüş}} - T_{\text{patlama}}, \\ T_{\text{yanıt}} &= T_{\text{ilk çalışma}} - T_{\text{geliş}}. \end{aligned}$$

Süreç G/Ç de yapıyorsa G/Ç'de geçen süre de çıkarılmalıdır: $T_{\text{bekleme}} = T_{\text{dönüş}} - T_{\text{patlama}} - T_{\text{G/Ç}}$; çünkü bekleme süresi yalnızca hazır kuyruğunda geçen süreyi sayar. Uzunluğu T olan ve işlemcinin B kadar meşgul olduğu bir zaman aralığında tamamlanan n süreç için

$$\text{işlemci kullanımı} = \frac{B}{T}, \qquad \text{üretilen iş} = \frac{n}{T}.$$



Şekil 10: Bir kez kesintiye uğrayan bir süreç: patlama $= 3 + 2 = 5$, bekleme $= 4 + 2 = 6$, dönüş $= 11$, yanıt $= 4$.

Formülleri şekil üzerinde sağlamak

$T_{\text{dönüş}} = 11 - 0 = 11$; $T_{\text{bekleme}} = 11 - 5 = 6$, ki bu gerçekten de hazır kuyruğunda geçen $4 + 2$ birimdir; $T_{\text{yanıt}} = 4 - 0 = 4$. Böyle bir süreç için yanıt $\le$ bekleme $\le$ dönüş. Kesintisiz zamanlamada bir süreç tek çalışmasından önce yalnızca bir kez bekler; bu yüzden yanıt süresi bekleme süresine eşittir.

6.2 Çözümlü örnek: geliş sırasına göre dört süreç

Tek bir işlemciye dört süreç geliyor. Onları “akla ilk gelen” sırayla, yani ilk gelen ilk hizmet alır (first come, first served, FCFS) kuralıyla ve kesinti olmadan çalıştırıyoruz. Bunun iyi bir sıra olup olmadığı 5. haftanın sorusudur; burada yalnızca tanımları alıştırma olarak uyguluyoruz. Tüm süreler milisaniyedir.

Süreç	Geliş	Patlama
P_1	0	5
P_2	1	3
P_3	2	6
P_4	16	2

Gantt şemasını kurmak. $t = 0$’da yalnızca P_1 vardır ve çalışır. P_2 ($t = 1$) ve P_3 ($t = 2$), P_1 çalışırken gelir ve hazır kuyruğunda bekler. $t = 5$’te P_1 biter ve önce gelmiş olan P_2, $t = 8$’e kadar çalışır; ardından P_3, $t = 14$’e kadar çalışır. Şimdi hazır kuyruğu boştur; bu yüzden işlemci P_4 $t = 16$’da gelene kadar boşta kalır; P_4 $t = 18$’e kadar çalışır.



Şekil 11: Çözümlü örneğin Gantt şeması (FCFS sırası).

Süreç	Geliş	Patlama	Tamamlanma	Dönüş	Bekleme	Yanıt
P_1	0	5	5	$5 - 0 = 5$	$5 - 5 = 0$	0
P_2	1	3	8	$8 - 1 = 7$	$7 - 3 = 4$	4
P_3	2	6	14	$14 - 2 = 12$	$12 - 6 = 6$	6
P_4	16	2	18	$18 - 16 = 2$	$2 - 2 = 0$	0
Ortalama				$26/4 = 6.5$	$10/4 = 2.5$	2.5

- Ortalama dönüş süresi $= (5+7+12+2)/4 = 6.5$ ms; ortalama bekleme süresi $= (0+4+6+0)/4 = 2.5$ ms; ortalama yanıt süresi $= 2.5$ ms (zamanlama kesintisiz olduğu için bekleme süresine eşit).
- İşlemci kullanımı $=$ 16 ms meşgul$/18$ ms $\approx 88.9\%$. Boşta geçen aralık kullanımı düşürür, ama hiç kimsenin bekleme süresini artırmaz.
- Üretilen iş $= 4/18$ ms ≈ 0.22 süreç/ms, yani saniyede yaklaşık 222 süreç.

6.3 İyileştirme hedefleri

Özetle işlemci kullanımını ve üretilen işi **en büyük**; dönüş, bekleme ve yanıt süresini **en küçük** yapmak isteriz. Genellikle *ortalama* değer iyileştirilir. Bazı durumlarda en küçük ya da en büyük değeri iyileştirmek daha iyidir; etkileşimli sistemlerde *öngörülebilir* (düşük varyanslı) bir yanıt süresi, düşük bir ortalamadan daha önemli olabilir: hep 50 ms süren bir tepki, ara sıra 2 s’ye çıkan 5 ms’lik bir tepkiden daha iyi hissettirir.

Bu hedefler çatışır ve farklı sistemler onlara farklı ağırlıklar verir:

- **Toplu iş sistemleri ve sunucular:** üretilen iş ve işlemci kullanımı.
- **Masaüstü bilgisayarlar ve telefonlar:** yanıt süresi ve öngörülebilirliği.
- **Gerçek zamanlı sistemler:** her son tarihe uymak, her türlü ortalamadan daha önemlidir.

Tipik bir ödünleşim: süreçler arasında daha sık geçiş yapmak yanıt süresini iyileştirir, ama her geçiş dağıtım gecikmesine mal olur; dolayısıyla üretilen iş düşer. Bu, 2. haftadaki verimlilik ile kullanım kolaylığı çatışmasının yeni bir biçimidir.

7 Özet

Yedi cümlede 4. hafta

1. İş parçacığı; kendi kimliği, program sayacı, yazmaçları ve yığıtı olan bir yürütme birimidir; bir sürecin iş parçacıkları onun kodunu, verisini, öbeğini ve açık dosyalarını paylaşır.
2. Çok iş parçacıklı programlama yanıt verebilirlik, kaynak paylaşımı, ekonomi (makinemizde iş parçacığı yaratmak süreç yaratmaktan yaklaşık 4 kat ucuzdu) ve çok çekirdekli işlemcilerde ölçeklenebilirlik sağlar.
3. Eşzamanlılık birkaç görevin ilerlemesi demektir; paralellik birkaç görevin aynı anda çalışması demektir ve birkaç çekirdek gerektirir.
4. Amdahl yasası, hızlanma $\leq 1/\big(S + (1-S)/N\big)$, seri kesir S'nin hızlanmayı $1/S$ ile sınırladığını gösterir.
5. Kullanıcı iş parçacıkları çekirdek iş parçacıklarına çoktan bire, birebir (Linux, Windows) ya da çoktan çoğa eşlenir; Pthreads `pthread_create()` ve `pthread_join()` çağrılarını sunar.
6. Süreçler CPU ve G/Ç patlamaları arasında gidip gelir; kısa vadeli zamanlayıcı dört tür olayda hazır kuyruğundan sıradaki süreci seçer; kesintili zamanlayıcılar çalışan bir süreci kesebilir de; geçişi dağıtıcı yapar.
7. Zamanlayıcılar işlemci kullanımı ve üretilen işle (en büyük) ve dönüş, bekleme ve yanıt süresiyle (en küçük) değerlendirilir.

8 Sözlük

Amdahl yasası (Amdahl's law) Seri kesri S olan bir programın N çekirdekteki hızlanması için üst sınır: $1/(S + (1-S)/N)$.

Bekleme süresi (waiting time) Bir sürecin hazır kuyruğunda geçirdiği toplam süre.

CPU patlaması / G/Ç patlaması (CPU burst / I/O burst) Bir sürecin hesap yaptığı / G/Ç beklediği dönem.

Çekirdek düzeyi iş parçacığı (kernel-level thread) İşletim sistemi çekirdeğinin yarattığı ve zamanladığı iş parçacığı.

Çoklu iş parçacığı modeli (multithreading model) Kullanıcı iş parçacıklarının çekirdek iş parçacıklarına eşlenmesi: çoktan bire, birebir ya da çoktan çoğa.

Dağıtıcı (dispatcher) İşlemciyi zamanlayıcının seçtiği sürece veren işletim sistemi modülü.

Dağıtım gecikmesi (dispatch latency) Dağıtıcının bir süreci durdurup başka birini başlatması için gereken süre.

Dönüş süresi (turnaround time) Tamamlanma zamanı eksi geliş zamanı.

Eşzamanlılık (concurrency) Birkaç görevin aynı dönem içinde, gerekirse tek çekirdekte sırayla, ilerlemesi.

Hazır kuyruğu (ready queue) Çalışmaya hazır olup bir işlemci bekleyen süreçler kümesi.

İş parçacığı (thread) İşlemcide yürütmenin birimi: iş parçacığı kimliği, program sayacı, yazmaçlar ve yığıt; gerisini süreciyle paylaşır.

İşlemci yoğun, G/Ç yoğun (CPU-bound, I/O-bound) Zamanının çoğunu hesaplayarak / G/Ç bekleyerek geçiren süreç.

Kesintili zamanlama (preemptive scheduling) İşletim sisteminin işlemciyi çalışan bir süreçten geri alabildiği zamanlama.

Kısa vadeli (işlemci) zamanlayıcı (short-term / CPU scheduler) İşletim sisteminin hazır kuyruğundan çalışacak sıradaki süreci seçen kısmı.

Kullanıcı düzeyi iş parçacığı (user-level thread) Kullanıcı alanındaki bir kütüphanenin yönettiği, çekirdeğin görmediği iş parçacığı.

Paralellik (parallelism) Birkaç görevin farklı çekirdeklerde aynı anda yürütülmesi.

Pthreads İş parçacığı yaratmak ve yönetmek için POSIX standart API'si.

Üretilen iş (throughput) Birim zamanda tamamlanan süreç sayısı.

Yanıt süresi (response time) Gelişten, sürecin işlemciyi ilk kez aldığı ana kadar geçen süre.

9 Kendiniz Deneyin

Kendi makinenizde iş parçacıkları ve zamanlama

Linux (laboratuvar bilgisayarları, WSL ya da bir sanal makine), `code/` klasörünün içinde:

```
$ nproc                          # kaç çekirdek?
$ make                           # dört programı da derle
$ ./array_sum 1; ./array_sum 2; ./array_sum 4
$ ./thread_hello & ps -eLf | grep thread_hello
$ time ./burst cpu ; time ./burst io
$ vmstat 1 5                     # cs sütunu = saniyedeki bağlam değişimi
```

Windows: Görev Yöneticisi → *Ayrıntılar* sekmesinde bir sütun başlığına sağ tıklayın, *Sütunları seç*'i seçin ve *İş parçacıkları*'nı ekleyin. Görev Yöneticisi → *Performans* → *CPU*, çekirdek ve mantıksal işlemci sayısını gösterir.

1. Makinenizde kaç çekirdek var? 2, 4 ve 8 iş parçacığıyla hangi hızlanmayı elde ediyorsunuz?
2. İki iş parçacığıyla elde ettiğiniz hızlanmadan Amdahl yasasıyla S'yi tahmin edin.
3. Bilgisayarınızda en çok iş parçacığına sahip program hangisi?

4. Bir sayfayı okurken tarayıcınız işlemci yoğun mu, G/Ç yoğun mu? Ya video oynatırken?

10 Tekrar Soruları

Kavramsal

1. İş parçacığını tanımlayın. Bir sürecin hangi kısımları iş parçacıklarınca paylaşılır, hangileri her iş parçacığına özeldir?
2. Çok iş parçacıklı programlamanın dört yararını, her birine bir örnekle verin.
3. Eşzamanlılık ile paralellik arasındaki farkı açıklayın. Tek çekirdekli bir bilgisayar eşzamanlı olabilir mi? Paralel olabilir mi?
4. Çok çekirdekli programlamanın beş zorluğunu sayın. Dizi toplama programı bunlardan hangisinden kaçınır ve nasıl?
5. Kullanıcı düzeyi ve çekirdek düzeyi iş parçacıklarını karşılaştırın: her birinin bir avantajı ve bir dezavantajı.
6. Çoktan bire bir iş parçacığı kütüphanesi neden ikinci bir çekirdeği kullanamaz? Linux ve Windows hangi modeli kullanır?
7. `thread_hello` programında neden tüm iş parçacıkları aynı PID'yi ama farklı TID'leri yazar? Satırların sırası neden değişebilir?
8. İşlemci–G/Ç patlama döngüsünü anlatın. İşlemci yoğun ve G/Ç yoğun süreçler nasıl farklılaşır ve `time` bu farkı nasıl ortaya koyar?
9. Bir zamanlama kararının verilebileceği dört durumu sayın. Bunların hangilerinde yeni bir süreç *seçilmelidir*?
10. Kesintili ve kesintisiz zamanlama arasındaki fark nedir? Kesinti neden yarış durumlarına yol açar?
11. Dağıtıcı ne yapar? Dağıtım gecikmesi nedir ve neden küçük olmalıdır?
12. Zamanlama algoritmasından en doğrudan etkilenen ölçüt neden bekleme süresidir?

Sayısal

13. Bir program %60 paraleldir ($S = 0.4$). 2, 4 ve 8 çekirdekteki en büyük hızlanmayı ve sonsuz sayıda çekirdek için sınırı hesaplayın.
14. Bir program tek çekirdekte 100 s çalışıyor; bunun 80 s'si kusursuzca paralelleştirilebiliyor. Program 8 çekirdekte ne kadar sürer ve hızlanma nedir?
15. 4 çekirdekte en az 3 kat hızlanmaya ihtiyacınız var. Buna izin veren en büyük seri kesir S nedir?
16. Bir web sunucusu saniyede 2000 istek alıyor. Ölçtüğümüz maliyetlerle (süreç yaratmak 205.5 μs, iş parçacığı yaratmak 51.9 μs), her istek için yalnızca bir süreç ya da bir iş parçacığı yaratmaya bir çekirdeğin ne kadarı harcanır?
17. Dört süreç kesintisiz FCFS ile çalışıyor: A (geliş 0, patlama 8), B $(2, 4)$, C $(4, 1)$, D $(6, 3)$; süreler ms. Gantt şemasını çizin; her sürecin dönüş ve bekleme süresini, ortalamaları, işlemci kullanımını ve üretilen işi hesaplayın.

18. Bir bağlam değişimi $5\ \mu$s sürüyor. Zaman dilimi (a) 10 ms, (b) 1 ms ise işlemci zamanının ne kadarı ek yüktür? (Her zaman diliminden sonra bir geçiş olduğunu varsayın.)

Sayısal soruların yanıtları

13. $N = 2$: $1/(0.4 + 0.3) \approx 1.43$; $N = 4$: $1/(0.4 + 0.15) \approx 1.82$; $N = 8$: $1/(0.4 + 0.075) \approx 2.11$; sınır $1/0.4 = 2.5$.

14. $T(8) = 20 + 80/8 = 30$ s; hızlanma $= 100/30 \approx 3.33$. (Burada $S = 0.2$: $1/(0.2 + 0.8/8) = 1/0.3 \approx 3.33$.)

15. $3 = 1/\big(S+(1-S)/4\big) \Rightarrow S+(1-S)/4 = 1/3 \Rightarrow 4S+1-S = 4/3 \Rightarrow 3S = 1/3 \Rightarrow S = 1/9 \approx 0.11$; yani en çok yaklaşık %11 seri.

16. Süreçler: $2000 \times 205.5 \times 10^{-6}$ s $= 0.411$ s işlemci zamanı/saniye, yani bir çekirdeğin %41,1'i. İş parçacıkları: $2000 \times 51.9 \times 10^{-6} \approx 0.104$ s, yani %10,4. (Gerçek sunucular bundan bile kaçınmak için iş parçacıklarını bir *iş parçacığı havuzundan* (thread pool) yeniden kullanır.)

17. Gantt şeması: A 0–8, B 8–12, C 12–13, D 13–16. Dönüş: $A = 8$, $B = 12 - 2 = 10$, $C = 13 - 4 = 9$, $D = 16 - 6 = 10$; ortalama $37/4 = 9.25$ ms. Bekleme: $A = 0$, $B = 10 - 4 = 6$, $C = 9 - 1 = 8$, $D = 10 - 3 = 7$; ortalama $21/4 = 5.25$ ms. İşlemci hiç boşta kalmaz: kullanım %100; üretilen iş $4/16 = 0.25$ süreç/ms. (Kısa süreç C'nin 1 ms'lik bir patlama için 8 ms beklediğine dikkat edin — 5. hafta için bir ipucu.)

18. (a) $5/(10\,000 + 5) \approx 0.05\%$; (b) $5/(1000 + 5) \approx 0.50\%$. On kat kısa bir zaman dilimi on kat fazla ek yük getirir.

11 İleriye Bakış: 5. Hafta

- Zamanlama algoritmaları: ilk gelen ilk hizmet alır (FCFS), en kısa iş önce (SJF) ve onun kesintili biçimi SRTF, öncelikli zamanlama ve döngüsel sıralı zamanlama (round robin, RR).
- Her algoritma için, tam olarak bu haftanın ölçütlerini kullanarak Gantt şemaları ve ortalama bekleme ve dönüş süreleri.
- Süreç eşzamanlaması: yarış durumları (bizim `shared_counter`++ örneğimiz) ve kritik bölge problemi (critical-section problem).
- Muteks kilitleri (mutex locks) ve semaforlar.

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley, Bölüm 4 (Threads & Concurrency, Kısım 4.1–4.4) ve Bölüm 5 (CPU Scheduling, Kısım 5.1–5.2).
- W. Stallings, *Operating Systems: Internals and Design Principles*, 9. baskı, Pearson, Bölüm 4 ve 9.
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Bölüm 26, “Concurrency: An Introduction”: `https://pages.cs.wisc.edu/~remzi/OSTEP/threads-intro.pdf`
  - Bölüm 27, “Interlude: Thread API”: `https://pages.cs.wisc.edu/~remzi/OSTEP/threads-api.pdf`
  - Bölüm 7, “Scheduling: Introduction”: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf`